

Searching algorithms: These are essential tools in computer science used to locate specific items within a collection of data. In this tutorial, we are mainly going to focus upon searching in an array. When we search an item in an array, there are two most common algorithms used based on the type of input array.

- **Linear Search:** It is used for an unsorted array. It mainly does one by one comparison of the item to be search with array elements. It takes linear or $O(n)$ Time.
- **Binary Search:** It is used for a sorted array. It mainly compares the array's middle element first and if the middle element is same as input, then it returns. Otherwise it searches in either left half or right half based on comparison result (Whether the mid element is smaller or greater). This algorithm is faster than linear search and takes $O(\log n)$ time.

Linear Search Algorithm: In Linear Search, we iterate over all the elements of the array and check if it the current element is equal to the target element. If we find any element to be equal to the target element, then return the index of the current element. Otherwise, if no element is equal to the target element, then return -1 as the element is not found. Linear search is also known as sequential search.

Working:

- 1.Start from the first element of the list.
- 2.Compare the current element with the target value.
- 3.If they match → return the index (or say “found”).
- 4.If not → move to the next element.
- 5.Repeat until:
 - The element is found, or
 - The end of the list is reached (in which case the element isn't in the list).

Applications of Linear Search Algorithm:

- **Unsorted Lists:** When we have an unsorted array or list, linear search is most commonly used to find any element in the collection.
- **Small Data Sets:** Linear Search is preferred over binary search when we have small data sets with
- **Searching Linked Lists:** In linked list implementations, linear search is commonly used to find elements within the list. Each node is checked sequentially until the desired element is found.
- **Simple Implementation:** Linear Search is much easier to understand and implement as compared to Binary Search or Ternary Search.

Advantages of Linear Search Algorithm:

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

Disadvantages of Linear Search Algorithm:

- Linear search has a time complexity of $O(N)$, which in turn makes it slow for large datasets.
- Not suitable for large arrays.

When to use Linear Search Algorithm?

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

Binary Search Algorithm: Binary Search is an efficient algorithm used to find the position of a target element in a sorted array. It works by repeatedly dividing the search interval in half.

Working:

1. Start with the whole array.
2. Find the middle element.
3. Compare the middle element with the target:
 - If it matches → element found.
 - If target < middle → search in the left half.
 - If target > middle → search in the right half.
4. Repeat steps 2–3 until the element is found or the array ends.

Advantages:

- 1. Very fast for large datasets:** Time complexity is $O(\log n)$.
- 2. Predictable performance:** Works efficiently regardless of array size.
- 3. Less comparisons:** Compared to Linear Search, it requires far fewer checks.

Disadvantages:

- 1. Requires sorted data:** Cannot be used on unsorted arrays.
- 2. More complex implementation:** Harder to code than Linear Search.
- 3. Not suitable for linked lists:** Random access is needed for efficiency.

Applications:

- 1. Searching in large sorted datasets:** Like databases, phone directories, or libraries.
- 2. Finding boundaries or ranges:** Ex: lower bound or upper bound in arrays.
- 3. Algorithms in computer science:** Used in binary search trees, search optimization, and coding problems.
- 4. Real-time searching:** When speed is critical and data is sorted.

Sorting Algorithm: A Sorting Algorithm is used to rearrange a given array or list of elements in an order. For example, a given array [10, 20, 5, 2] becomes [2, 5, 10, 20] after sorting in increasing order and becomes [20, 10, 5, 2] after sorting in decreasing order.

Bubble Sort: Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

Working:

1. We sort the array in several passes:

- After the first pass, the largest element moves to the end (its correct position).
- After the second pass, the second largest element moves to the second last position, and so on.

2. In each pass, we only look at the unsorted part of the array.

3. We compare adjacent elements and swap them if the first one is larger than the second.

4. By repeating this process, the largest element in the unsorted part moves to its correct position at the end of that pass.

Advantages of Bubble Sort:

1. Bubble sort is easy to understand and implement.

2. It does not require any additional memory space.

3. It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort:

1. Bubble sort has a time complexity of $O(n^2)$ which makes it very slow for large data sets.

2. Bubble sort has almost no or limited real world applications. It is mostly used in academics to teach different ways of sorting.

Applications:

1. Sorting small datasets: Useful for small lists where performance doesn't matter much.

2. Educational purposes: Commonly used to teach sorting logic and algorithm basics.

3. Nearly sorted data: Works efficiently when only a few elements are out of order.

4. Simple embedded systems: Used in systems with low memory or simple processing needs.

Selection Sort: Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.

Working:

1. First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.

2. Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.

3. We keep doing this until we get all elements moved to correct position.

Advantages of Selection Sort:

1. Works well for small datasets: Efficient enough when the number of elements is small.

2. Fewer swaps (memory writes): It makes the minimum number of swaps among simple sorting algorithms, which is useful when writing to memory is costly.

3. No extra memory needed: It sorts the list in place, using only constant extra space ($O(1)$).

Disadvantages of the Selection Sort

- 1.Inefficient for large datasets:** Time complexity is always $O(n^2)$, even if the array is already sorted.
- 2.Not stable:** Equal elements may change their relative order after sorting.
- 3.Still makes comparisons for every element:** Even if part of the array is sorted, it keeps comparing all remaining elements.

Applications of Selection Sort:

- 1.Perfect for teaching fundamental sorting mechanisms and algorithm design.
2. Works well for sorting small lists of numbers, names, or records.
- 3.Heap Sort algorithm is based on Selection Sort.

Insertion Sort: Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.

Working:

- 1.We start with the second element of the array as the first element is assumed to be sorted.
- 2.Compare the second element with the first element if the second element is smaller then swap them.
- 3.Move to the third element, compare it with the first two elements, and put it in its correct position
- 4.Repeat until the entire array is sorted.

Advantages:

- 1.Simple and easy to implement.
- 2.Stable sorting algorithm.
- 3.Efficient for small lists and nearly sorted lists.
- 4.Space-efficient as it is an in-place algorithm.

Disadvantages:

- 1.Inefficient for large lists.
- 2.Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

Application:

- 1.Small datasets:** Very efficient for sorting small arrays or lists.
- 2.Nearly sorted data:** Performs extremely well when the array is almost sorted, because it moves elements only when needed.
- 3.Online sorting / real-time applications:** Useful when data arrives one by one, and you need to keep it sorted as it comes (like real-time ranking).

Merge Sort: Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Working:

- 1.Divide:** Divide the list or array recursively into two halves until it can no more be divided.
- 2.Conquer:** Each subarray is sorted individually using the merge sort algorithm.
- 3.Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Advantages:

- 1.Efficient for large datasets:** Time complexity is always $O(n \log n)$.
- 2.Stable sorting:** Preserves the relative order of equal elements.
- 3.Works well for linked lists:** No extra shifting required.
- 4.Predictable performance:** Doesn't depend on the initial order of elements.

Disadvantages:

- 1.Requires extra memory:** Needs $O(n)$ space for temporary arrays.
- 2.More complex implementation:** Harder to code compared to simple sorts like Bubble or Insertion Sort.
- 3.Slower for small datasets:** For small arrays, simple algorithms may be faster due to lower overhead

Applications:

- 1.Large datasets:** Sorting big arrays efficiently.
- 2.External sorting:** Used in files and databases that don't fit in memory.
- 3.Linked lists:** Very effective because no shifting is needed.
- 4Stable sorting required:** When maintaining the original order of equal elements matters.

Quick Sort: This is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array. It works on the principle of divide and conquer, breaking down the problem into smaller sub-problems.

Working:

- 1.Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
- 2.Partition the Array:** Re arrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then in its correct position, and we obtain the index of the pivot.
- 3.Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays (left and right of the pivot).
- 4.Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Advantages of Quick Sort:

- 1.It is a divide-and-conquer algorithm that makes it easier to solve problems.
- 2.It is efficient on large data sets.
- 3.It has a low overhead, as it only requires a small amount of memory to function.

Disadvantages of Quick Sort

1. It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
2. It is not a good choice for small data sets.

Applications of Quick Sort:

1. Sorting large datasets efficiently in memory.
2. Used in library sort functions (like C++ `std::sort` and Java `Arrays.sort` for primitives).
3. Arranging records in databases for faster searching.

Linked List: A linked list is a fundamental data structure in computer science. It mainly allows efficient insertion and deletion operations compared to arrays. Like arrays, it is also used to implement other data structures like stack, queue and deque.

Singly Linked List: A singly linked list is a fundamental data structure, it consists of nodes where each node contains a data field and a reference to the next node in the linked list. The next of the last node is null, indicating the end of the list. Linked Lists support efficient insertion and deletion operations.

Advantage:

1. Dynamic size (no fixed limit like arrays).
2. Efficient insertion and deletion at beginning and end. We also have insertion at the middle efficient if we have reference or pointer to the node after which we need to insert.
3. Can implement complex data structures like stack, queue, graph.

Disadvantage:

1. Extra memory required for storing pointers.
2. No direct/random access (need traversal).
3. Cache unfriendly (not stored in contiguous memory).

Doubly Linked List: A doubly linked list is a type of linked list where each node has two pointers: one to the next node and one to the previous node.

Advantages:

1. **Traversal in both directions:** Can move forward and backward easily.
2. **Easier insertion and deletion:** No need to traverse from the head for deletion of a previous node.
3. **Better flexibility:** Useful for implementing complex data structures like **deques, stacks, and queues**.
4. **Efficient for certain operations:** Faster operations when you need access to both previous and next nodes.

Disadvantages:

- 1.More memory usage:** Each node stores an extra pointer (previous node).
- 2.More complex implementation:** Harder to code and manage compared to singly linked lists.
- 3.Higher chance of errors:** Extra pointers increase the risk of mistakes like broken links.

Circular Singly Linked List: In Circular Singly Linked List, each node has just one pointer called the "next" pointer. The next pointer of the last node points back to the first node and this results in forming a circle. In this type of Linked list, we can only move through the list in one direction.

Circular Doubly Linked List: In circular doubly linked list, each node has two pointers prev and next, similar to doubly linked list. The prev pointer points to the previous node and the next points to the next node. Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.

Advantage of Circular Linked List:

- 1.Efficient Traversal.
- 2.No Null Pointers / References.
- 3.Useful for Repetitive Tasks.
- 4.Insertion at Beginning or End is $O(1)$.
- 5.Uniform Traversal.
- 6.Efficient Memory Utilization.

Disadvantage of Circular Linked List:

- 1.Complex Implementation.
- 2.Infinite Loop Risk.
- 3.Harder to Debug.
- 4.Deletion Complexity.
- 5.Memory Overhead (for Doubly Circular LL).
- 6.Not Cache Friendly.

Table for Time and space Complexity for both Sorting and Searching Algorithm:**1.Sorting Algorithms:**

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

2.Searching Algorithms:

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Notes
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Works on unsorted array
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Requires sorted array

Abstract Data Type (ADT):

- 1.An Abstract Data Type (ADT) is a concept or model that describes what operations can be done on data - but not how those operations are carried out.
- 2.It focuses on what a data structure should do, rather than how it is implemented or how the data is stored in memory.
- 3.It's called "abstract" because it hides the details of implementation and only shows the important operations and behaviors.

Operations:

1.Basic (or Primitive) Operations: These are the main actions that define what the ADT can do; they change or use the data.

Examples:

ADT	Basic Operations
Stack	push(x), pop(), peek()
Queue	enqueue(x), dequeue(), front()
List	insert(x), delete(x), get(i)
Set	add(x), remove(x), contains(x)
Map (Dictionary)	put(key, value), get(key), remove(key)

2.Helper (or Query) Operations: These check or return information about the data without changing it.

Examples:

- i. **isEmpty():** checks if the structure is empty
- ii. **isFull():** checks if it's full (in fixed-size structures)
- iii. **size():** returns the number of elements
- iv. **display():** shows all elements

Features of ADT:

- 1.Abstraction:** The user does not need to know the implementation of the data structure only essentials are provided.
- 2.Information Hiding:** ADTs can protect the integrity of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
- 3.Modularity:** ADTs can be combined with other ADTs to form larger, more complex data structures. This allows for greater flexibility and modularity in programming.
- 4.Robust:** The program is robust and has the ability to catch errors.

Data Structures: A data structure is a way of organizing and storing data so that it can be used efficiently. Data structures can be classified into two main types:

- Linear Data Structures
- Non-Linear Data Structures

3.Linear Data Structures: In linear data structures, data elements are arranged one after another in a sequence. Each element has a unique predecessor and successor (except the first and last).

Examples of Linear Data Structures

Data Structure	Description	Example
Array	Fixed-size collection of elements of the same type	[10, 20, 30, 40]
Linked List	Collection of nodes where each node points to the next	10 → 20 → 30 → NULL
Stack	Follows LIFO (Last In, First Out) order	Plates stacked one on top of another
Queue	Follows FIFO (First In, First Out) order	People standing in a line

Example of Linear Order: 10 → 20 → 30 → 40; Each element is connected in a straight sequence.

Non-Linear Data Structures: In non-linear data structures, data elements are not arranged in a sequence. Each element can be connected to multiple elements, forming a hierarchy or network.

Examples of Non-Linear Data Structures:

Data Structure	Description	Example
Tree	Hierarchical structure with root and child nodes	Family tree, file directory
Graph	Collection of nodes (vertices) connected by edges	Social network, road map

Example of Non-Linear Structure: Here, A is connected to B and C, showing a hierarchical relationship.

```
A
 / \
B   C
 / \
D   E
```

Algorithm: An algorithm is a step-by-step procedure or a set of instructions designed to solve a specific problem or perform a particular task. It tells a computer exactly what to do, in what order, to get the desired result.

Characteristics:

- 1.Clear and Unambiguous:** The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- 2.Well-Defined Inputs:** If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- 3.Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- 4.Finite-ness:** The algorithm must be finite, i.e. it should terminate after a finite time.
- 5.Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
- 6.Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.

NOTE: Not all procedures are algorithms. But every algorithm is a procedure.

Properties of Algorithm:

- 1.It should terminate after a finite time.
- 2.It should produce at least one output.
- 3.It should take zero or more input.
- 4.Every step in the algorithm must be effective i.e. every step should do some work.

Advantages of Algorithms:

- 1.It is easy to understand.
- 2.An algorithm is a step-wise representation of a solution to a given problem.
- 3.In an Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms:

- 1.Writing an algorithm takes a long time so it is time-consuming.
- 2.Understanding complex logic through algorithms can be very difficult.

Algorithm Complexity: Algorithm complexity is a measure of the efficiency of an algorithm in terms of the time it takes to run and the memory (space) it uses as the input size increases. It helps us understand how an algorithm's performance changes with larger inputs, allowing us to compare and choose the most efficient solution for a problem.

Types of Algorithm Complexity:

1.Time Complexity: Measures how much time an algorithm takes to complete its execution.

2.Space Complexity: Measures how much memory an algorithm needs during execution.

1.Space Complexity: Space Complexity is the amount of memory (space) an algorithm needs to run to completion. It includes all the memory used by:

- Input data
- Variables
- Temporary storage
- Function calls (stack space)
- Data structures created during execution

Two components:

1.Fixed Space: Memory needed for constants, program code, and fixed-size variables.

2.Variable Space: Memory needed for input data, temporary variables, recursion stack, and dynamic structures (like arrays or linked lists).

Why Space Complexity Matters:

- 1.It helps us understand how much memory a program will use.
- 2.Important for systems with limited RAM (like embedded devices or mobile apps).
- 3.Allows us to compare algorithms that solve the same problem but use different amounts of memory.

2.Time Complexity: Time Complexity is a measure of how much time an algorithm takes to complete its execution, depending on the size of the input (n). It helps us understand how fast or slow an algorithm runs as the input grows.

Why It Matters:

- 1.Helps compare algorithms that solve the same problem.
- 2.Allows us to predict performance for large inputs.
- 3.Helps us write efficient programs that run faster.

Types of Analysis:

1.Best Case Analysis: This is the minimum time an algorithm takes to run. It happens when the input is ideal (the easiest or most favorable case).

Ex: In linear search, if the element you're looking for is at the first position, then the algorithm stops immediately.

→ **Best case:** $O(1)$

2.Worst Case Analysis: This is the maximum time an algorithm can take to run. It happens in the most difficult situation or when the input is unfavorable. It helps us ensure that the algorithm will never be slower than this bound.

Ex: In linear search, if the element is not in the list or is at the last position, the algorithm checks all elements.

→ **Worst case:** $O(n)$

3.Average Case Analysis: This is the expected time an algorithm takes to run on average, considering all possible inputs. It gives a more realistic idea of performance in normal use.

Ex: In linear search, if the element could be anywhere in the list, on average, half the elements are checked.

→ **Average case:** $O(n/2) \approx O(n)$

Asymptotic Notation: Asymptotic Notation is a way to describe the efficiency of an algorithm in terms of time or space complexity as the input size (n) becomes very large.

It focuses on the growth rate of an algorithm, ignoring constants and smaller terms that don't affect performance for large inputs.

Why It's Used:

- 1.Helps compare algorithms independent of hardware or implementation.
- 2.Shows efficiency for large input sizes.
- 3.Ignores constant factors and low-order terms to focus on dominant behavior.

Types of Notations:

1.Big O Notation (O): Big O Notation is used to describe the upper bound of an algorithm's running time. It tells us the maximum time an algorithm could take to complete, given the size of the input.

How it Measures the Upper Bound: Big O notation focuses on the worst-case scenario. It helps us understand the maximum time or space an algorithm will require, regardless of the input's specifics.

Ex:

$O(1)$: Constant time. The algorithm takes the same amount of time regardless of the input size.

Example: Accessing an element in an array.

$O(n)$: Linear time. The algorithm's running time grows linearly with the input size. Example: Iterating through a list.

$O(n^2)$: Quadratic time. The running time grows quadratically as the input size increases. Example: Bubble sort.

Linear Search (element not found):

```
for(int i = 0; i < n; i++) {  
    if(arr[i] == key)  
        return i;  
}
```

- Worst case: the element is not in the array → loop runs n times
- Time Complexity: $O(n)$

2. Omega Notation (Ω): Omega Notation is used to describe the lower bound of an algorithm's running time. It tells us the minimum time an algorithm will take to complete, given the size of the input.

How it Measures the Lower Bound: Omega notation focuses on the best-case scenario. It helps us understand the least amount of time or space an algorithm will require.

Ex:

$\Omega(1)$: Constant time. The algorithm takes at least a constant amount of time, regardless of the input size.

$\Omega(n)$: Linear time. The algorithm takes at least linear time as the input size grows.

Linear Search (element at first position):

```
for(int i = 0; i < n; i++) {  
    if(arr[i] == key)  
        return i; // Found immediately  
}
```

- Best case: the element is first $\rightarrow$ only 1 iteration
- Time Complexity: $\Omega(1)$

3. Theta Notation (Θ): Theta Notation is used to describe the exact bound of an algorithm's running time. It tells us the average-case scenario, where the running time is bound both above and below by the same function.

How it Measures the Exact Bound: Theta notation focuses on the typical running time of an algorithm. It shows that the algorithm's running time is both at least and at most a certain function of the input size.

Ex:

$\Theta(1)$: Constant time. The algorithm takes a constant amount of time, regardless of the input size.

$\Theta(n)$: Linear time. The algorithm's running time grows linearly with the input size.

Simple loop of n iterations:

```
for(int i = 0; i < n; i++) {  
    cout << i << " ";  
}
```

- Loop runs exactly n times $\rightarrow$ both upper and lower bound are the same
- Time Complexity: $\Theta(n)$